Curso de Assimilação de Dados para Previsão Numérica de Tempo e Clima

Paulo Yoshio Kubota

Curso de Assimilação de Dados Atmosféricos

Carga horária sugerida: 40–60 horas

Formato: Aulas teóricas + exercícios práticos (Python/Fortran) + estudo de casos reais

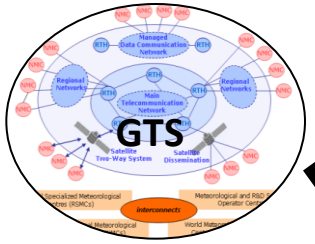
Módulo 1 — Fundamentos

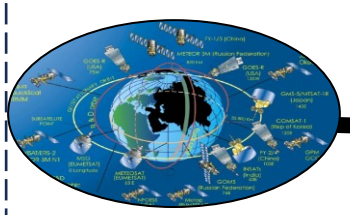
Módulo 1 — Fundamentos

1. **Motivação e aplicações**
    1. Previsão numérica do tempo (NWP)
    2. Monitoramento climático
    3. Reanálises atmosféricas
2. **O que é assimilação de dados**
    1. Conceito: combinar observações e modelos
    2. Diferença para interpolação pura
    3. Exemplos: GFS, ECMWF, CPTEC
3. **Fontes de dados**
    1. Satélites (radiâncias, ventos derivados, perfis de temperatura/umidade)
    2. Radar meteorológico
    3. Estações meteorológicas
    4. Radiossondas
4. **Revisão de conceitos básicos**
    1. Equações da dinâmica de fluidos
    2. Conceito de estado do sistema
    3. Erros de modelo e de observação

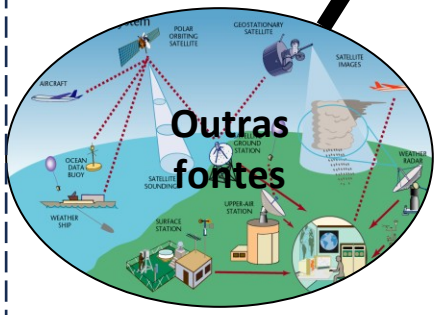
Introdução "O que será abordado"?







Nicolas R. Dalezios



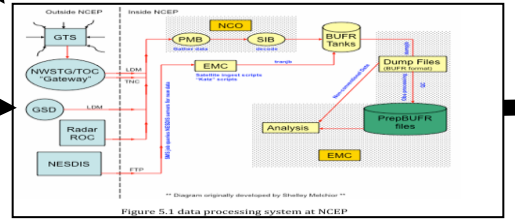
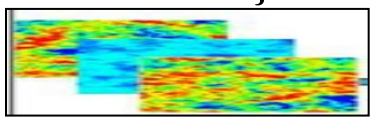


Figure 5.1 data processing system at NCEP

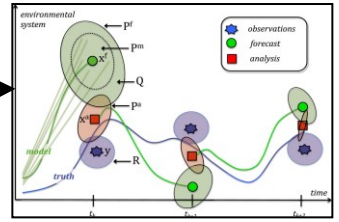
Pré-processamento
BUFR/PrepBUFR

Inicialização



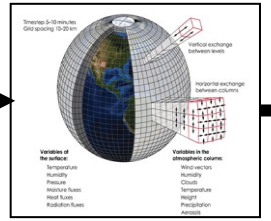
First Guess ($_{anl,fct,clm,..}$)

GSI



$$T_a = T_b + \frac{\sigma_b^2}{\sigma_b^2 + \sigma_o^2}(T_o - T_b)$$

Sem balanço



MODELO NUMERICO

balanceado

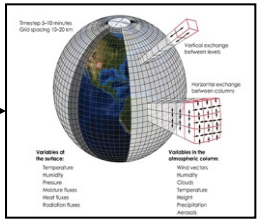
(forecasting reference 3 or 6 h)

Assimilação da dados

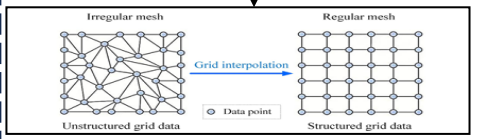
Análise



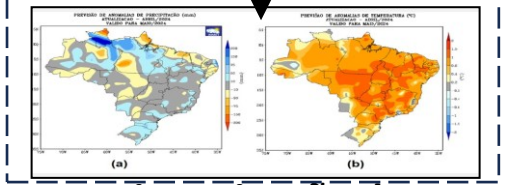
MODELO NUMERICO



forecasting



Pós-processamento



Disseminação de Produtos

https://community.wmo.int/en/GTS_WIS2_Transition_Guidance

Danel Mashakova

Paulo Yoshio Kubota

Módulo 1 — Fundamentos

1. Motivação e Aplicações

1.1 Previsão Numérica do Tempo (NWP)

•Modelos numéricos que simulam a evolução da atmosfera são usados para prever o tempo, como o :

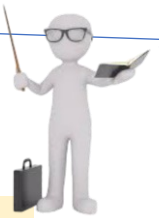
- **GFS** (Global Forecast System)
- **ECMWF** (European Centre for Medium-Range Weather Forecasts),
- **BAM** (CPTEC/INPE),

Equações Diferenciais Parciais Para Simulações Atmosféricas?

$\vec{F} = m\vec{a}$

$\Delta U = \Delta Q - W$

Base para PNTC



$fluxo = q = \frac{quantidade\ atravessando\ um\ contorno}{area\ do\ contorno\ \boldsymbol{X}\ duração\ do\ tempo}$

•Equação da continuidade

Estrutura da atmosfera Estática Obedece a hidrostática

Segue a lei da Equação de estado para os gases ideais

•Equações de Navier-Stokes (forma primitiva)

$$\frac{Du_i}{Dt} = \frac{\partial u_i}{\partial t} + u_j\frac{\partial u_i}{\partial x_j} = -\frac{1}{\rho_0}\frac{\partial P}{\partial x_i} - \frac{\rho}{\rho_0}g\delta_{i3} - 2\Omega\varepsilon_{ijk}\eta_j u_k + \nu\left(\frac{\partial^2 u_i}{\partial {x_j}^2}\right)$$

•Equação de energia (termodinâmica)

$$\frac{\partial T}{\partial t} + (\vec{V}.\nabla)T - S_P\omega = \frac{J}{C_p}$$

•Equação da conservação de massa

$$\frac{\partial \rho}{\partial t} + \nabla\cdot\overrightarrow{\rho V} = \frac{S}{\Delta x\Delta y\Delta z} = \frac{S}{V}$$

•Equação da continuidade

$$\frac{\partial \rho}{\partial t} + \nabla\cdot\overrightarrow{\rho V} = 0$$

$P = -\rho g z$

$P = \rho RT$

Paulo Yoshio Kubota

As Equações EDPs “Governantes do Estado Médio da

Equações Conservação de Momentum **(F=ma)**

$$\frac{\partial(\bar{u})}{\partial t}+(\bar{u})\frac{\partial(\bar{u})}{\partial x}+(\bar{v})\frac{\partial(\bar{u})}{\partial y}+(\bar{w})\frac{\partial(\bar{u})}{\partial z}+\frac{1}{\rho_0}\frac{\partial(\bar{P})}{\partial x}-2\Omega\eta_3(\bar{v})-\nu\frac{\partial^2(\bar{u})}{\partial x^2}-\nu\frac{\partial^2(\bar{u})}{\partial y^2}-\nu\frac{\partial^2(\bar{u})}{\partial z^2}=-\frac{\partial(\overline{u'u'})}{\partial x}-\frac{\partial(\overline{v'u'})}{\partial y}-\frac{\partial(\overline{w'u'})}{\partial z}$$

$$\frac{\partial(\bar{v})}{\partial t}+(\bar{u})\frac{\partial(\bar{v})}{\partial x}+(\bar{v})\frac{\partial(\bar{v})}{\partial y}+(\bar{w})\frac{\partial(\bar{v})}{\partial z}+\frac{1}{\rho_0}\frac{\partial(\bar{P})}{\partial y}+2\Omega\eta_3(u)-\nu\frac{\partial^2(\bar{v})}{\partial x^2}-\nu\frac{\partial^2(\bar{v})}{\partial y^2}-\nu\frac{\partial^2(v)}{\partial z^2}=-\frac{\partial(\overline{u'v'})}{\partial x}-\frac{\partial(\overline{v'v'})}{\partial y}-\frac{\partial(\overline{w'v'})}{\partial z}$$

$$\frac{\partial(\bar{w})}{\partial t}+(\bar{u})\frac{\partial(\bar{w})}{\partial x}+(\bar{v})\frac{\partial(\bar{w})}{\partial y}+(\bar{w})\frac{\partial(\bar{w})}{\partial z}+\frac{1}{\rho_0}\frac{\partial(\bar{P})}{\partial z}+g\frac{\bar{\rho}}{\rho_0}-\nu\frac{\partial^2(\bar{w})}{\partial x^2}-\nu\frac{\partial^2(\bar{w})}{\partial y^2}-\nu\frac{\partial^2(\bar{w})}{\partial z^2}=-\frac{\partial(\overline{u'w'})}{\partial x}-\frac{\partial(\overline{v'w'})}{\partial y}-\frac{\partial(\overline{w'w'})}{\partial z}$$

Equação Conservação de Energia **(DU=DQ-DW)**

$$\frac{\partial(\bar{T})}{\partial t}+(\bar{u})\frac{\partial(\bar{T})}{\partial x}+(\bar{v})\frac{\partial(\bar{T})}{\partial y}+(\bar{w})\frac{\partial(\bar{T})}{\partial z}-S_P\bar{\omega}=-\frac{\partial(\overline{u'T'})}{\partial x}-\frac{\partial(\overline{v'T'})}{\partial y}-\frac{\partial(\overline{w'T'})}{\partial z}+\frac{\bar{J}}{C_p}$$

$$\frac{\rho}{\rho_0}=\frac{kg_{H_20}}{kg_{ar}}=\bar{c}$$



$$u_3=w$$
$$x_3=z$$
$$\nu\frac{\partial^2(\overline{u_3})}{\partial {x_3}^2}\cong 0$$

Equação Conservação de Massa (eq continuidade $\frac{\partial\rho}{\partial t}+\frac{\partial\rho\vec{V}}{\partial x_i}=\bar{S}_c\Rightarrow\frac{\partial\rho}{\partial t}+\vec{V}\frac{\partial\rho}{\partial x_i}=\bar{S}_c-\rho\frac{\partial\vec{V}}{\partial x_i}$)

$$\frac{\partial(\bar{c})}{\partial t}+(\bar{u})\frac{\partial(\bar{c})}{\partial x}+(v)\frac{\partial(\bar{c})}{\partial y}+(\bar{w})\frac{\partial(\bar{c})}{\partial z}=-\frac{\partial(\overline{u'c'})}{\partial x}-\frac{\partial(\overline{u'c'})}{\partial y}-\frac{\partial(\overline{w'c'})}{\partial z}+\bar{S}_c$$

$$P=\rho RT$$

$$P=\rho gH$$

CI, CC, INT-NL, FIS, APROX

EDPs / CI e CC / Problema Bem postado/ Mal postado

Problema Bem postado

CI, CC, INT-NL, FIS, APROX

Um problema de condição inicial / contorno bem postado tem uma solução única que depende continuamente das condições iniciais / contorno.

Para problemas mal postado, pequenos erros nas condições iniciais / de contorno podem produzir erros enormes na solução.

A *especificação das* condições iniciais e contorno adequadas para uma EDPs é essencial para ter um problema bem colocado.

- **Se *muitas condições* iniciais / contorno (incertas) forem especificadas, não haverá solução.**
- **Se *poucas condições* iniciais / contorno forem especificados, a solução não será exclusiva.**
- **Se o *número de condições* iniciais / contorno estiver certo , mas forem especificadas no lugar ou hora errada, a solução será única, mas não dependerá somente do amortecimento das condições iniciais / contorno .**

Paulo Yoshio Kubota

Módulo 1 — Fundamentos

•O **objetivo da assimilação de dados** é melhorar a previsão, ***ajustando o estado inicial do modelo com base nas observações*** reais.

Módulo 1 — Fundamentos

1. Motivação e Aplicações

1.2 Monitoramento Climático

•A assimilação de dados também é usada para **reanálises climáticas**, ajustando modelos para representar melhor as condições atmosféricas passadas. Exemplos incluem reanálises como **ERA5**.

1.3 Reanálises Atmosféricas

•Utiliza-se a assimilação de dados para fornecer uma representação mais precisa das condições atmosféricas passadas, levando em conta tanto os dados de observação quanto os modelos.

Módulo 1 — Fundamentos

2. O que é Assimilação de Dados?

Definição: A assimilação de dados é o **processo de ajustar um modelo numérico para que ele se aproxime** o mais possível das observações reais.

O objetivo é melhorar a previsão do tempo e os modelos climáticos ajustando seu **estado com as observações**.

Módulo 1 — Fundamentos

2.1 Diferença entre Interpolação e Assimilação de Dados

Interpolação: A **interpolação é simplesmente a estimativa de valores em pontos** onde não há observações, com base em pontos conhecidos. Não leva em conta a evolução do modelo

Assimilação de Dados: A assimilação **combina informações de modelos e observações**, e ajusta o estado do sistema de forma a otimizar a previsão.

Módulo 1 — Fundamentos

3. Fontes de Dados

As **principais fontes de dados** utilizadas em assimilação de dados atmosféricos são:

Satélites: Medições de radiância (como o METEOSAT ou o GOES) e perfis de temperatura e umidade.

Radar Meteorológico: Dados de reflectividade para monitorar precipitação.

Estações Meteorológicas: Medições diretas de temperatura, umidade, pressão, e vento.

Radiossondas: Perfis verticais de temperatura, umidade e pressão atmosférica.

Módulo 1 — Fundamentos

3. Fontes de Dados

slides

MET-348-3_Curso_de_Assimilacao_de_Dados_Modulo_0_PGMET

Módulo 1 — Fundamentos

4. Revisão de Conceitos Básicos

Equações da Dinâmica de Fluidos: O sistema atmosférico é modelado como um fluido dinâmico. As equações de Navier-Stokes são a base, embora em **modelos de previsão simplificados**, seja comum *usar uma versão linearizada.*

Estado do Sistema: O "**estado**" da atmosfera em um dado momento é *representado por um vetor* $\vec{x}$, que contém variáveis como a temperatura, pressão, vento e umidade em diversos pontos da grade do modelo.

Módulo 1 — Fundamentos

Erros de Modelo e Observação:

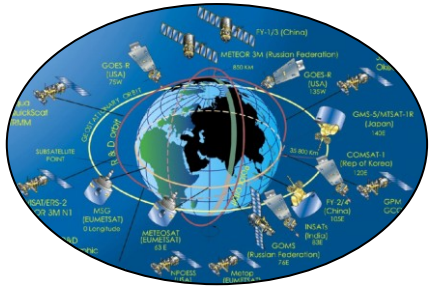
Erro de Modelo : Desvios devido a simplificações ou parametrizações.

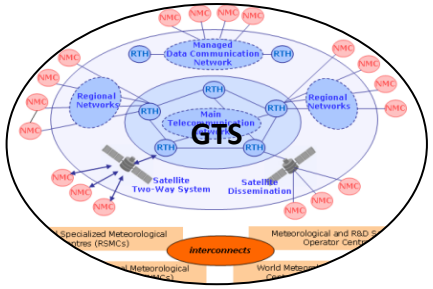
$$\frac{Du_i}{Dt} = \frac{\partial u_i}{\partial t} + u_j \frac{\partial u_i}{\partial x_j} = -\frac{1}{\rho_0}\frac{\partial P}{\partial x_i} - \frac{\rho}{\rho_0} g\delta_{i3} - 2\Omega\varepsilon_{ijk}\eta_j u_k + \nu\left(\frac{\partial^2 u_i}{\partial {x_j}^2}\right)$$

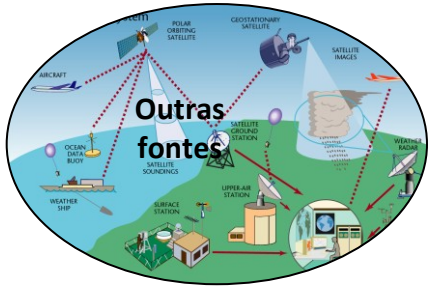
$$\frac{\partial T}{\partial t} + (\vec{V}.\nabla)T - S_P\omega = \frac{J}{C_p}$$

$$\frac{\partial \rho}{\partial t} + \nabla \cdot \overrightarrow{\rho V} = \frac{S}{\Delta x \Delta y \Delta z} = \frac{S}{V}$$

Erro de Observação: Desvios devido a limitações dos instrumentos.







Paulo Yoshio Kubota

“Etapas em uma previsão determinística do tempo”

Roteiro

- Sistemas globais de observação e assimilação de dados

Métodos de assimilação de dados

- Interpolação Ótima
- Assimilação de dados variacional 3D e 4D
- Métodos do Filtro de Kalman
- Nudging

- Assimilação de dados e teoria de estimação:
- Há vários critérios que são usados para estimar parâmetros e funções:
  - Estimação dos mínimos quadrados
  - Mínimos quadrados com restrições
  - Máxima verossimelhança
  - Mínima variância

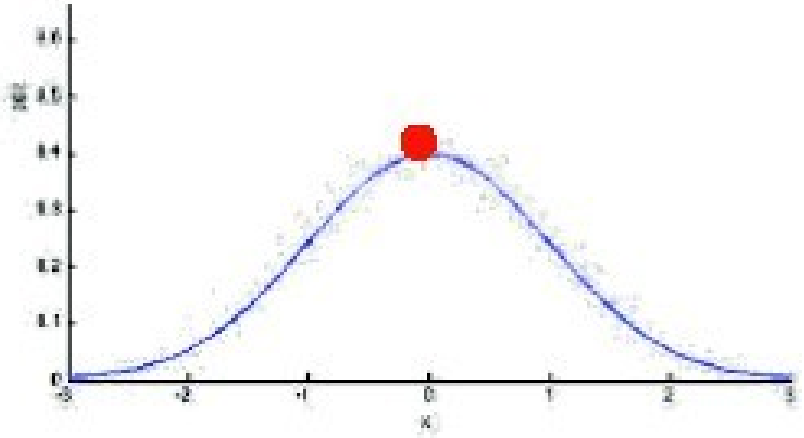
Métodos

SEQUENCIAL

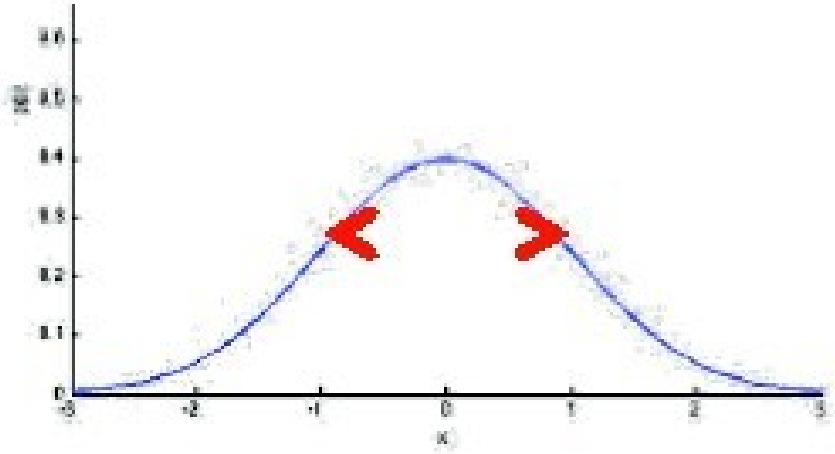
- Interpolação ótima
- EKF, EnKF

VARIACIONAL

- 3D (espaço)
- 4D (3d + tempo)



HIBRIDO



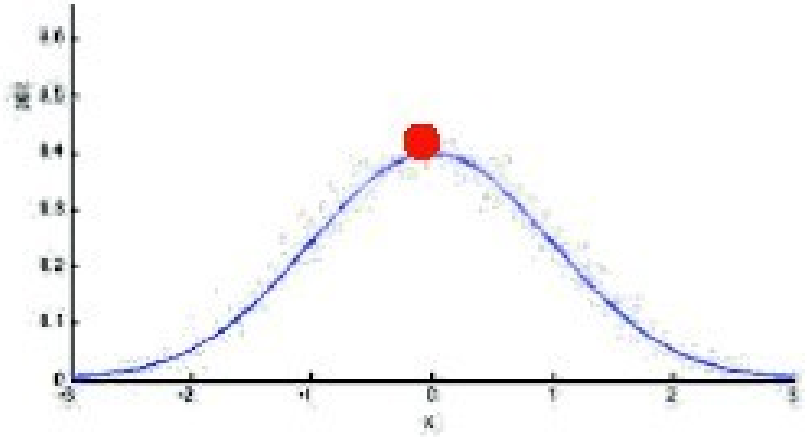
Métodos

$$w = \bar{w} + w'$$

$$\theta = \bar{\theta} + \theta'$$

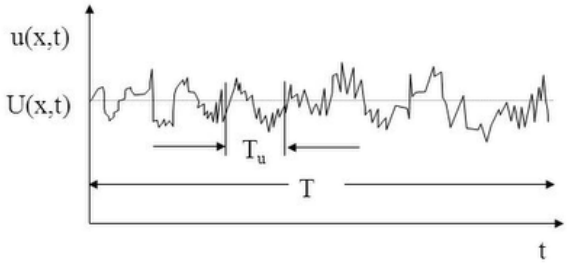
SEQUENCIAL

- Interpolação ótima
- EKF, EnKF



Exemplo I

- Como estimar de forma ótima a verdadeira temperatura T_t (análise) da sala?
  - Temos duas informações:



$$T_1 = T_t + \varepsilon_1$$
$$T_2 = T_t + \varepsilon_2$$

First guess $\varepsilon_1 = (T_1 - T_t)$

Observação $\varepsilon_2 = (T_2 - T_t)$

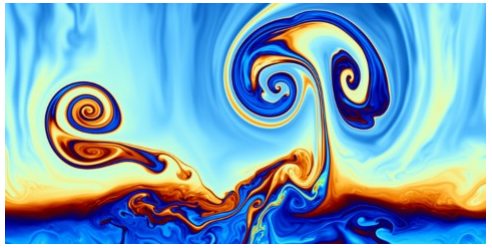
  - onde:

$$\overline{\varepsilon_1} = \overline{\varepsilon_2} = 0$$

Erros não tendenciosos

$$\overline{\varepsilon_1^2} = \sigma_1^2 \qquad \overline{\varepsilon_2^2} = \sigma_2^2$$

Variâncias



$$\overline{\varepsilon_1 \varepsilon_2} = \overline{(T_1 - T_t)}\ \overline{(T_2 - T_t)} = 0$$

Não correlacionados

Combinação linear

$$T_a = (a_1 T_1 + a_2 T_2)$$

Erros não tendenciosos

$$\overline{T_a} = \overline{T_t}$$

verdadeira temperatura T_t

Então

$$a_1 + a_2 = 1$$

A melhor estimativa de T_a de T_t será quando escolhemos os coeficientes a_1 e a_2 minimizando o erro médio quadrado de T_a ou seja:

Erros não tendenciosos $\varepsilon_1 = (T_1 - T_t)$ $\varepsilon_2 = (T_2 - T_t)$

$$\overline{\varepsilon_a^2} = \overline{(T_a - T_t)^2}$$

Variâncias $\overline{\varepsilon_1^2} = \sigma_1^2$ $\overline{\varepsilon_2^2} = \sigma_2^2$

$$\overline{\varepsilon_a^2} = \overline{\sigma_a^2}$$

$\overline{\varepsilon_1^2} = \overline{(T_1 - T_t)^2}$ $\overline{\varepsilon_2^2} = \overline{(T_2 - T_t)^2}$

$$\overline{\varepsilon_1^2} = \sigma_1^2 \qquad \overline{\varepsilon_2^2} = \sigma_2^2$$

Não correlacionados

$$T_a = (a_1 T_1 + a_2 T_2)$$

$$\sigma_a^2 = \varepsilon_a{}^2 = \overline{(a_1\varepsilon_1 + a_2\varepsilon_2)^2}$$

$$\varepsilon_1 = (T_1 - T_t)$$

$$\varepsilon_2 = (T_2 - T_t)$$

$$\sigma_a^2 = \overline{(T_a - T_t)^2} = \overline{[a_1(T_1 - T_t) + a_2(T_2 - T_t)]^2}$$

$$\sigma_a^2 = \overline{(T_a - T_t)^2} = \overline{[a_1 T_1 + a_2 T_2 - a_1 T_t - a_2 T_t]^2}$$

$$\sigma_a^2 = \overline{(T_a - T_t)^2} = \overline{[a_1 T_1 - a_2 T_t - a_1 T_t + a_2 T_2]^2}$$

$$\sigma_a^2 = \overline{(T_a - T_t)^2} = \overline{[a_1 T_1 - a_1 T_t + (1 - a_1) T_2 - (1 - a_1) T_t]^2}$$

$$\sigma_a^2 = \overline{(T_a - T_t)^2} = \overline{\left(a_1(T_1 - T_t) + (1 - a_1)(T_2 - T_t)\right)^2}$$

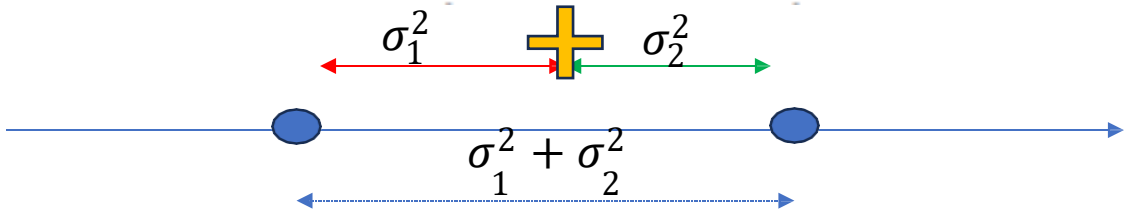
$$\sigma_a^2 = \overline{(T_a - T_t)^2} = \overline{\left(a_1(T_1 - T_t) + (1 - a_1)(T_2 - T_t)\right)^2}$$

$$a_2 = 1 - a_1$$

$\varepsilon_1 = (T_1 - T_t)$ $\quad$ $\varepsilon_2 = (T_2 - T_t)$

$$\sigma_a^2 = \overline{(T_a - T_t)^2} = \overline{[a_1(T_1 - T_t) + (1 - a_1)(T_2 - T_t)]^2}$$

$\overline{\varepsilon_1^2} = \sigma_1^2$ $\quad$ $\overline{\varepsilon_2^2} = \sigma_2^2$



$$a_1 + a_1 = 1$$

$$a_1 = \frac{\sigma_2^2}{\sigma_1^2 + \sigma_2^2} \quad a_2 = \frac{\sigma_1^2}{\sigma_1^2 + \sigma_2^2}$$

ou

$$a_1 = \frac{1/\sigma_1^2}{1/\sigma_1^2 + 1/\sigma_2^2} \quad a_2 = \frac{1/\sigma_2^2}{1/\sigma_1^2 + 1/\sigma_2^2}$$

$$T_a = (a_1 T_1 + a_2 T_2)$$

Paulo Yoshio Kubota

Assumindo que $T_1=T_b$ (background) e $T_2=T_o$ (observação)

$$a_1 + a_2 = 1 \qquad a_1 = 1 - a_2$$

$$T_a = a_1 T_b + a_2 T_o = T_b + a_2 (T_o - T_b)$$

$$T_a = T_b + \frac{\sigma_b^2}{\sigma_b^2 + \sigma_o^2}(T_o - T_b) = T_b + w(T_o - T_b)$$

Analise é uma previsão e observação otimamente combinadas

$$\overline{\varepsilon_a^2} = \overline{(T_a - T_0)^2}$$

$$\overline{\varepsilon_a^2} = \overline{\sigma_a^2}$$

$$T_a = T_b + \frac{\sigma_b^2}{\sigma_b^2 + \sigma_o^2}(T_o - T_b) \rightarrow \frac{1}{\sigma_a^2} = \frac{1}{\sigma_b^2} + \frac{1}{\sigma_o^2}$$

$$T_a = T_b + \frac{\sigma_b^2}{\sigma_b^2 + \sigma_o^2}(T_o - T_b)$$

Extensão

$$T_b \rightarrow \mathbf{x}_b; \quad T_a \rightarrow \mathbf{x}_a; \quad y_o \rightarrow \mathbf{y}_o;$$

$$\sigma_b^2 \rightarrow \mathbf{B}; \quad \sigma_a^2 \rightarrow \mathbf{A}; \quad \sigma_o^2 \rightarrow \mathbf{R};$$

$$\mathbf{x}_b(t_{i+1}) = M\left[\mathbf{x}_a(t_i)\right]$$ Modelo de previsão

$$T_a = T_b + \frac{\sigma_b^2}{\sigma_b^2 + \sigma_o^2}(T_o - T_b)$$

$$\mathbf{x}_a = \mathbf{x}_b + \mathbf{K}\left[\mathbf{y}_o - H(\mathbf{x}_b)\right]$$

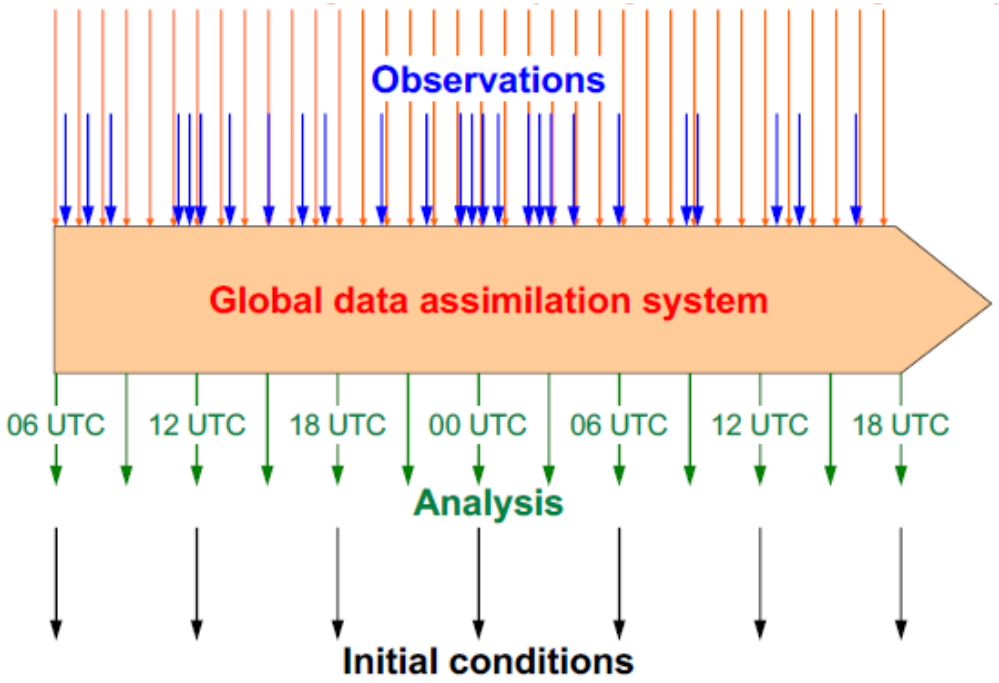
$$\mathbf{x}_a = \mathbf{x}_b + \mathbf{K}\left[\mathbf{y}_o - H(\mathbf{x}_b)\right]$$

Objetivo:
Derivar os algoritmos seqüenciais (IO/KF) e variacionais (3D-VAR, 4D-VAR)

“Etapas em uma previsão determinística do tempo”

“Assimilação global de dados”

“Previsão de curto prazo anterior (primeira estimativa ou estado de referência)”



Os **sistemas globais de assimilação de dados** recebem uma ampla gama de dados de diversos instrumentos e horários de observação.

“Assimilação: uma técnica de análise que acumula as informações observadas no modelo.”

“Análise: conjunto de dados espacialmente e temporalmente coerente em uma grade, baseado nas observações na medida do possível.”

A assimilação executa um **modelo de circulação geral (GCM)** em modo de retroprevisão (hindcast).
A análise resultante é uma descrição **espacialmente e temporalmente coerente** do estado real da atmosfera.

Módulo 1 — Fundamentos

Em regiões com **escassez de dados**, onde poucas observações estão disponíveis, esses sistemas, essencialmente, fornecem uma **mistura entre uma previsão de curto prazo** (por exemplo, 6 horas) e as observações disponíveis.

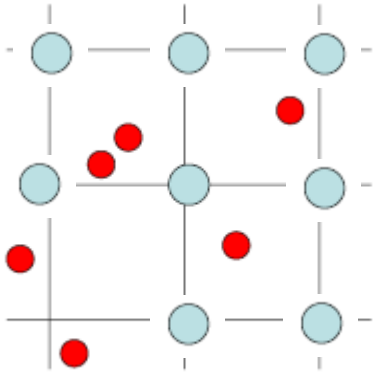
Exemplo Prático: Interpolação Ótima (OI)

Objetivo: Vamos usar um **exemplo simples de assimilação de dados** por ***interpolação ótima*** para ajustar um modelo com base em observações.

Exemplo de Dados:
Temos um modelo com um **valor estimado de temperatura em diferentes pontos da grade** e **uma observação de temperatura em um desses pontos**.

O modelo e a observação ***têm erros de estimativa***.

Interpolação Ótima (OI)



- Observações de altura geopotencial (z_i^{obs}, z_i^{fg})
- Ponto de grade (z_a, z_b^{fg}) , onde $z_a = analise$

Aqui, z_b^{fg} se refere à **primeira estimativa (ou estado de referência)**, que é derivada de uma **previsão de curto prazo anterior**.

Premissa 1: A **análise** é definida como uma **média ponderada sobre as observações vizinhas:**

$$z_a = z_b^{fg} + \sum_{i=1}^{m} A_i\left(z_i^{obs} - z_i^{fg}\right)$$

Aqui, z_i^{fg} é a **interpolação de z_b^{fg} para a localização da observação**.

Premissa 2: Os **pesos** A_i são escolhidos de forma que o **erro médio ao longo de muitos casos seja minimizado** (no sentido de **mínimos quadrados**). Para isso, o seguinte funcional deve ser minimizado:

$$J = \frac{1}{2}\langle (z^t - z_a)^2 \rangle = \left\langle \frac{1}{2}\left[z^t - z^{fg} - \sum_{i=1}^{m} A_i\left(z_i^{obs} - z_i^{fg}\right)\right]^2 \right\rangle$$

onde z^t define o **valor verdadeiro no ponto de grade**, e
onde os **colchetes externos** indicam a **média ao longo de algum período de tempo**.

Paulo Yoshio Kubota

Interpolação Ótima (OI)

Premissa 2: Os **pesos** A_i são escolhidos de forma que o **erro médio ao longo de muitos casos seja minimizado** (no sentido de **mínimos quadrados**). Para isso, o seguinte funcional deve ser minimizado:

$$J = \frac{1}{2}\langle (z^t - z)^2 \rangle = \left\langle \frac{1}{2}\left[z^t - z^{fg} - \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \right]^2 \right\rangle$$

$$\frac{\partial J}{\partial A_l} = \frac{\partial}{\partial A_l} \left\langle \frac{1}{2}\left[z^t - z^{fg} - \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \right]^2 \right\rangle$$

$$\frac{\partial J}{\partial A_l} = \left\langle \frac{2}{2}\left[z^t - z^{fg} - \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \right]^1 * \frac{\partial}{\partial A_l}\left[z^t - z^{fg} - \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \right] \right\rangle$$

$$\frac{\partial J}{\partial A_l} = \left\langle \left[z^t - z^{fg} - \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \right] * \left[\frac{\partial}{\partial A_l} z^t - \frac{\partial}{\partial A_l} z^{fg} - \frac{\partial}{\partial A_l} \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \right] \right\rangle$$

$$\frac{\partial J}{\partial A_l} = \left\langle \left[z^t - z^{fg} - \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \right] * \left[- \left\{ \frac{\partial A_i}{\partial A_l} \sum_{i=1}^{m} \left(z_i^{obs} - z_i^{fg} \right) + A_i \frac{\partial}{\partial A_l} \sum_{i=1}^{m} \left(z_i^{obs} - z_i^{fg} \right) \right\} \right] \right\rangle$$

Interpolação Ótima (OI)

Premissa 2: Os **pesos** A_i são escolhidos de forma que o **erro médio ao longo de muitos casos seja minimizado** (no sentido de **mínimos quadrados**). Para isso, o seguinte funcional deve ser minimizado:

$$\frac{\partial J}{\partial A_l} = \left\langle \left[z^t - z^{fg} - \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \right] * \left[- \left\{ \frac{\partial A_i}{\partial A_l} \sum_{i=1}^{m} \left(z_i^{obs} - z_i^{fg} \right) + A_i \frac{\partial}{\partial A_l} \sum_{i=1}^{m} \left(z_i^{obs} - z_i^{fg} \right) \right\} \right] \right\rangle$$

$$\frac{\partial J}{\partial A_l} = \left\langle \left[z^t - z^{fg} - \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \right] \left[- \left\{ \frac{\partial A_i}{\partial A_l} \sum_{i=1}^{m} \left(z_i^{obs} - z_i^{fg} \right) \right\} \right] \right\rangle$$

Se $A_i = A_l$

$$\frac{\partial J}{\partial A_l} = \left\langle \left[z^t - z^{fg} - \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \right] \left[- \left\{ \frac{\partial A_l}{\partial A_l} \sum_{l=1}^{m} \left(z_l^{obs} - z_l^{fg} \right) \right\} \right] \right\rangle$$

$$\frac{\partial J}{\partial A_l} = \left\langle \left[z^t - z^{fg} - \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \right] \left[- \left\{ \sum_{l=1}^{m} \left(z_l^{obs} - z_l^{fg} \right) \right\} \right] \right\rangle$$

Interpolação Ótima (OI)

Premissa 2: Os **pesos** A_i são escolhidos de forma que o **erro médio ao longo de muitos casos seja minimizado** (no sentido de **mínimos quadrados**). Para isso, o seguinte funcional deve ser minimizado:

$$\frac{\partial J}{\partial A_l} = \left\langle \left[z^t - z^{fg} - \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \right] \left[- \left\{ \sum_{l=1}^{m} \left(z_l^{obs} - z_l^{fg} \right) \right\} \right] \right\rangle$$

Para encontrar máximos, mínimos e pontos de inflexão, você deve calcular a primeira derivada da função, igualá-la a zero e encontrar os valores de x que satisfazem essa equação.

"A minimização fornece:"

$$\frac{\partial J}{\partial A_l} = \left\langle \left[z^t - z^{fg} - \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \right] \left[- \left\{ \sum_{l=1}^{m} \left(z_l^{obs} - z_l^{fg} \right) \right\} \right] \right\rangle = 0$$

Interpolação Ótima (OI)

Premissa 2: Os **pesos** A_i são escolhidos de forma que o **erro médio ao longo de muitos casos seja minimizado** (no sentido de **mínimos quadrados**). Para isso, o seguinte funcional deve ser minimizado:

$$\frac{\partial J}{\partial A_l} = \left\langle \left[z^t - z^{fg} - \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \right] \left[- \left\{ \sum_{l=1}^{m} \left(z_l^{obs} - z_l^{fg} \right) \right\} \right] \right\rangle = 0$$

$$\frac{\partial J}{\partial A_l} = - \left\langle \left[z^t - z^{fg} - \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \right] \right\rangle = \frac{0}{\left[- \left\{ \sum_{l=1}^{m} \left(z_l^{obs} - z_l^{fg} \right) \right\} \right]}$$

$$- \left\langle \left[z^t - z^{fg} - \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \right] \right\rangle = 0$$

"Converter em um sistema linear de equações para os pesos A_i" multiplique por $\left(z_l^{obs} - z_l^{fg} \right)$

$$- \left\langle \left[\left(z^t - z^{fg} \right) \left(z_l^{obs} - z_l^{fg} \right) - \sum_{i=1}^{m} A_i \left(z_i^{obs} - z_i^{fg} \right) \left(z_l^{obs} - z_l^{fg} \right) \right] \right\rangle = 0$$

Interpolação Ótima (OI)

Premissa 2: Os **pesos** A_i são escolhidos de forma que o **erro médio ao longo de muitos casos seja minimizado** (no sentido de **mínimos quadrados**). Para isso, o seguinte funcional deve ser minimizado:

$$-\left\langle\left[\left(z^t - z^{fg}\right)\left(z_l^{obs} - z_l^{fg}\right) - \sum_{i=1}^{m} A_i\left(z_i^{obs} - z_i^{fg}\right)\left(z_l^{obs} - z_l^{fg}\right)\right]\right\rangle = 0$$

$$\left\langle\left(z^t - z^{fg}\right)\left(z_l^{obs} - z_l^{fg}\right)\right\rangle - \sum_{i=1}^{m}\left\langle\left(z_i^{obs} - z_i^{fg}\right)\left(z_l^{obs} - z_l^{fg}\right)\right\rangle A_i = 0$$

Convert into a linear system of equations for weights A_l

$$\underbrace{\left\langle\left(z^t - z^{fg}\right)\left(z_l^{obs} - z_l^{fg}\right)\right\rangle}_{B_l} - \sum_{i=1}^{m}\underbrace{\left\langle\left(z_i^{obs} - z_i^{fg}\right)\left(z_l^{obs} - z_l^{fg}\right)\right\rangle}_{C_{l,i}} A_i = 0$$

Solve linear system for weights A_i

$$B_l - \sum_{i=1}^{m} C_{l,i} A_i = 0$$

B_l and $C_{l,i}$ known => can be solved for A_i

Paulo Yoshio Kubota

Interpolação Ótima (OI)

Premissa 2: Os **pesos** A_i são escolhidos de forma que o **erro médio ao longo de muitos casos seja minimizado** (no sentido de **mínimos quadrados**). Para isso, o seguinte funcional deve ser minimizado:

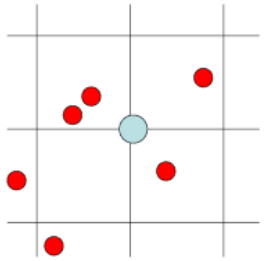
Convert into a linear system of equations for weights A_l

$$\underbrace{\left\langle \left(z^t - z^{fg}\right)\left(z_l^{obs} - z_l^{fg}\right)\right\rangle}_{B_l} - \sum_{i=1}^{m} \underbrace{\left\langle \left(z_i^{obs} - z_i^{fg}\right)\left(z_l^{obs} - z_l^{fg}\right)\right\rangle}_{C_{l,i}} A_i = 0$$

Solve linear system for weights A_i

$$B_l - \sum_{i=1}^{m} C_{l,i} A_i = 0$$

B_l and $C_{l,i}$ known => can be solved for A_i



- Observações de altura geopotencial (z_i^{obs}, z_i^{fg})
- Ponto de grade (z_a, z_b^{fg}) , onde $z_a = analise$

onde z^t define o **valor verdadeiro no ponto de grade (analise)**

B_l representa a **correlação (ou covariância) dos erros da primeira estimativa na localização da análise e de uma observação vizinha i**.

Ela envolve o **valor verdadeiro em um ponto de grade**, que não é conhecido.
Para derivar B_i, algumas **suposições simplificadoras** são necessárias (próximo slide).

B_l é às vezes referido como **"erro de análise"**.

Interpolação Ótima (OI)

Premissa 2: Os **pesos** A_i são escolhidos de forma que o **erro médio ao longo de muitos casos seja minimizado** (no sentido de **mínimos quadrados**). Para isso, o seguinte funcional deve ser minimizado:

Convert into a linear system of equations for weights A_l

$$\underbrace{\left\langle \left(z^t - z^{fg}\right)\left(z_l^{obs} - z_l^{fg}\right)\right\rangle}_{B_l} - \sum_{i=1}^{m} \underbrace{\left\langle \left(z_i^{obs} - z_i^{fg}\right)\left(z_l^{obs} - z_l^{fg}\right)\right\rangle}_{C_{l,i}} A_i = 0$$

Solve linear system for weights A_i

$$B_l - \sum_{i=1}^{m} C_{l,i} A_i = 0$$

B_l and $C_{l,i}$ known => can be solved for A_i

$\boldsymbol{C_{l,i}}$ representa a **correlação das diferenças entre $\boldsymbol{OBS}$ e $\boldsymbol{FG}$ nas observações vizinhas**.

Elas podem ser **calculadas a partir do desempenho de previsões passadas**.

A matriz $\boldsymbol{C_{l,i}}$ é chamada de **"matriz de covariância do erro da primeira estimativa"** (ou **"matriz de covariância do erro de referência"**).

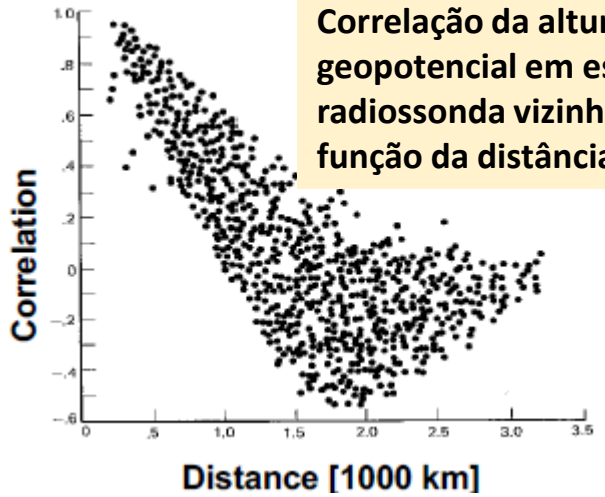
“Como escolher os pesos na Interpolação Ótima (OI)”

(1) Estatísticas das observações:

Para estimar o **“erro de análise”**, pode-se assumir que as respectivas **covariâncias**, ou seja:

$$B_l = \left\langle \left(z^t - z^{fg}\right)\left(z_l^{obs} - z_l^{fg}\right)\right\rangle$$

dependem apenas da distância entre as observações.



Correlação da altura geopotencial em estações de radiossonda vizinhas em função da distância.

Convert into a linear system of equations for weights A_l

$$\underbrace{\left\langle \left(z^t - z^{fg}\right)\left(z_l^{obs} - z_l^{fg}\right)\right\rangle}_{B_l} - \sum_{i=1}^{m} \underbrace{\left\langle \left(z_i^{obs} - z_i^{fg}\right)\left(z_l^{obs} - z_l^{fg}\right)\right\rangle}_{C_{l,i}} A_i = 0$$

Solve linear system for weights A_i

$$B_l - \sum_{i=1}^{m} C_{l,i} A_i = 0$$

B_l and $C_{l,i}$ known => can be solved for A_i

(2) Desempenho passado do sistema de Interpolação Ótima (OI):

Para **avaliar a covariância do erro da primeira estimativa** entre observações vizinhas, ou seja:

$$\boldsymbol{C_{l,i}} = \left\langle \left(z_i^{obs} - z_i^{fg}\right)\left(z_l^{obs} - z_l^{fg}\right)\right\rangle$$

utiliza-se informação do desempenho passado do sistema de assimilação.

Combinar esses dois tipos de informação permite escolher de forma objetiva os pesos ótimos A_i para a análise.

O procedimento é **complexo** e envolve o **desempenho passado de um sistema antigo de assimilação de dados OI**, bem como a **solução de um conjunto linear de equações**.

Paulo Yoshio Kubota

“Como escolher os pesos na Interpolação Ótima (OI)”

Exercício 1:

1.Implemente a **Interpolação Ótima (OI)** em Python, ajustando o valor de temperatura de um modelo baseado em uma observação com erro.

2.Modifique os erros do modelo e de observação e observe como isso afeta o valor de análise (temperatura ajustada).

3.Tente comparar os resultados com diferentes valores para o erro de observação.

$$z_a = z_b^{fg} + \sum_{i=1}^{m} A_i\left(z_i^{obs} - z_i^{fg}\right)$$

$$z_b^{fg} = \sum_{i=1}^{m} z_i^{fg}$$

$$z_a = \sum_{i=1}^{m}\left[z_i^{fg} + A_i\left(z_i^{obs} - z_i^{fg}\right)\right]$$

$$z_a = \sum_{i=1}^{m}\left(1 - A_i\right) z_i^{fg} + \sum_{i=1}^{m} A_i\left(z_i^{obs}\right)$$

```
!=========================================================
! Programa: interpolacao_otima.f90
! Autor: Paulo Kubota
!=========================================================

module class_interpolacao_otima_mod
   use iso_fortran_env, only: real64
   implicit none
   private
   public :: optimal_interpolation
contains

   pure function optimal_interpolation(x_model, y_model, x_obs, y_obs, model_error, obs_error)
result(y_analysis)
      ! Interpolacao otima simples (1D)
      real(real64), intent(in) :: x_model(:), y_model(:)
      real(real64), intent(in) :: x_obs(:), y_obs(:)
      real(real64), intent(in) :: model_error, obs_error
      real(real64) :: y_analysis(size(x_obs))

      real(real64) :: gain

      ! Ganho de interpolacao (simplificada)
      gain = model_error / (model_error + obs_error)

      ! Corrige o valor no ponto x = 2 ;  índice 3 (pois Fortran e 1-based)
      y_analysis = (1.0_real64 - gain) * y_model(3)    +    gain * y_obs

   end function optimal_interpolation

end module class_interpolacao_otima_mod
```

```
program interpolacao_otima
    use iso_fortran_env, only: real64
    use class_interpolacao_otima_mod
    implicit none

    real(real64), dimension(6) :: x_model, y_model
    real(real64), dimension(1) :: x_obs, y_obs, y_analysis
    real(real64) :: model_error, obs_error

    ! Dados do modelo
    x_model = [0.0, 1.0, 2.0, 3.0, 4.0, 5.0]
    y_model = [10.0, 12.0, 14.0, 16.0, 18.0, 20.0]

    ! Observacao
    x_obs = [2.0]
    y_obs = [15.5]

    ! Erros
    model_error = 1.0
    obs_error   = 0.5

    ! Aplicar Interpolacao Otima
    y_analysis = optimal_interpolation(x_model, y_model, x_obs, &
                                       y_obs, model_error, obs_error)

    ! Saída dos resultados
    write(*,'(A)') '========================================='
    write(*,'(A)') '      INTERPOLACAO OTIMA (OI) - FORTRAN   '
    write(*,'(A)') '========================================='
    write(*,'(A,F6.2)') 'Ponto observado (x = ', x_obs(1)
    write(*,'(A,F6.2)') 'Temperatura observada: ', y_obs(1)
    write(*,'(A,F6.2)') 'Temperatura do modelo: ', y_model(3)
    write(*,'(A,F6.2)') 'Temperatura apos assimilacao: ', y_analysis(1)
    write(*,'(A)') '========================================='

end program interpolacao_otima
```

```python
import numpy as np
import matplotlib.pyplot as plt

# Dados do modelo (temperatura estimada)
x_model = np.array([0  , 1  , 2   , 3  , 4  , 5  ])
y_model = np.array([10, 12, 14, 16, 18, 20])

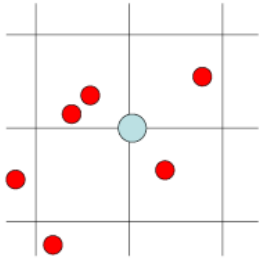
# Observação
x_obs = np.array([2])  # Ponto onde temos uma observação
y_obs = np.array([15.5])  # Observação real de temperatura no ponto x = 2

# Erro do modelo e erro de observação
model_error = 1.0  # Desvio padrão do erro do modelo
obs_error = 0.5  # Desvio padrão do erro de observação

# Interpolação Ótima
def optimal_interpolation(x_model, y_model, x_obs, y_obs, model_error, obs_error):
    # Calculando a função de análise
    gain = model_error / (model_error + obs_error)  # Ganha de interpolação
    y_analysis = (1 - gain) * y_model[2] + gain * y_obs  # Temperatura corrigida no ponto de observação
    return y_analysis

# Aplicando Interpolação Ótima
y_analysis = optimal_interpolation(x_model, y_model, x_obs, y_obs, model_error, obs_error)

# Visualizando o resultado
plt.plot(x_model, y_model, label='Modelo')
plt.scatter(x_obs, y_obs, color='red', label='Observação')
plt.scatter(x_obs, y_analysis, color='green', label='Análise (Após Assimilação)')
plt.legend()
plt.xlabel('Pontos na Grade')
plt.ylabel('Temperatura')
plt.title('Exemplo de Interpolação Ótima (OI)')
plt.show()
```



Exercício 1:

1.Implemente a **Interpolação Ótima (OI)** em Python, ajustando o valor de temperatura de um modelo baseado em uma observação com erro.

2.Modifique os erros do modelo e de observação e observe como isso afeta o valor de análise (temperatura ajustada).

3.Tente comparar os resultados com diferentes valores para o erro de observação.

- Observações de altura geopotencial (z_i^{obs}, z_i^{fg})
- Ponto de grade (z_a, z_b^{fg}) , onde $z_a = analise$

onde z^t define o **valor verdadeiro no ponto de grade (analise)**

Paulo Yoshio Kubota

```
!=====================================================================
! Programa : advec_diff_OI.f90
! Descrição: Modelo 1D de adveccao-difusao com assimilacao (OI)
! Autor    : Paulo Kubota
! Padrão   : Fortran 2023
!=====================================================================

module advec_diff_mod
   use iso_fortran_env, only: real64
   implicit none
contains
   pure function step_advec_diff(c, nx, dx, dt, u, kappa) result(c_new)
      ! Passo de tempo explicito da equacao de adveccao-difusao 1D
      integer, intent(in) :: nx
      real(real64), intent(in) :: dx, dt, u, kappa
      real(real64), intent(in) :: c(nx)
      real(real64) :: c_new(nx)
      integer :: i
      real(real64) :: adv, diff

      c_new = c
      do concurrent (i = 2:nx-1)
         adv  =  -u * (c(i) - c(i-1)) / dx
         diff =  kappa * (c(i+1) - 2.0_real64*c(i) + c(i-1)) / dx**2
         c_new(i) = c(i) + dt * (adv + diff)
      end do
   end function step_advec_diff
end module advec_diff_mod
```

```
program advec_diff_OI
   use iso_fortran_env, only: real64
   use advec_diff_mod
   implicit none
   ! ---------------------------
   ! Parametros do modelo
   ! ---------------------------
   integer, parameter :: nx = 50, nt = 50
   real(real64), parameter :: dx = 1.0_real64 / nx, dt = 0.001_real64
   real(real64), parameter :: u = 1.0_real64, kappa = 0.01_real64
   real(real64), allocatable :: x(:)
   real(real64), allocatable :: c_true(:,:), c_model(:,:), c_analysis(:,:)
   real(real64), allocatable :: c0(:)
   integer :: n, i
   integer, dimension(3) :: obs_idx = [11, 26, 41]  ! Fortran e 1-based
   real(real64) :: obs_error = 0.05_real64
   real(real64) :: model_error = 0.1_real64
   real(real64) :: gain
   real(real64), allocatable :: y_obs(:,:)
   real(real64) :: r
   ! ---------------------------
   ! Inicializacao
   ! ---------------------------
   allocate(x(nx), c_true(nt, nx), c_model(nt, nx), c_analysis(nt, nx))
   allocate(c0(nx), y_obs(nt, size(obs_idx)))

   ! Grade e condicao inicial (pulso gaussiano)
   do i = 1, nx
      x(i) = real(i-1, real64) / real(nx-1, real64)
      c0(i) = exp(-200.0_real64 * (x(i) - 0.3_real64)**2)
   end do

   c_true(1,:) = c0
   c_model(1,:) = 0.8_real64 * c0
```

```
! ----------------------------
! Simulacao da "verdade"
! ----------------------------
do n = 2, nt
   c_true(n,:) = step_advec_diff(c_true(n-1,:), nx, dx, dt, u, kappa)
end do
! ----------------------------
! Simulacao do "modelo"
! ----------------------------
do n = 2, nt
   c_model(n,:) = step_advec_diff(c_model(n-1,:), nx, dx, dt, u, kappa)
end do
! ----------------------------
! Geracao de observacoes (com ruido)
! ----------------------------
call random_seed()  ! inicializa gerador
do n = 1, nt
   do i = 1, size(obs_idx)
      call random_number(r)
      y_obs(n, i) = c_true(n, obs_idx(i)) + obs_error * (2.0_real64*r - 1.0_real64)
   end do
end do
! ----------------------------
! Interpolacao Otima (OI)
! ----------------------------
gain = model_error / (model_error + obs_error)
c_analysis = c_model
do n = 1, nt
   do i = 1, size(obs_idx)
      c_analysis(n, obs_idx(i)) = (1.0_real64 - gain)*c_model(n, obs_idx(i)) + gain*y_obs(n, i)
   end do
end do
! ----------------------------
```

```
 ! ----------------------------
   ! Saída final (ultimo passo)
   ! ----------------------------
   write(*,'(A)') '================================================='
   write(*,'(A)') ' Interpolacao Otima (OI) - Adveccao-Difusao 1D'
   write(*,'(A)') '================================================='
   write(*,'(A,F6.3)') 'Ganho = ', gain
   write(*,'(A)') 'Pontos observados e analise final:'
   do i = 1, size(obs_idx)
      write(*,'(A,I3,2(A,F6.3))') ' idx=', obs_idx(i), '  obs=', y_obs(nt,i), &
                                  '  analise=', c_analysis(nt, obs_idx(i))
   end do
   write(*,'(A)') '================================================='

   ! Exportar resultados (para plot posterior em Python ou Gnuplot)
   open(unit=10, file='oi_output.dat', status='replace')
   write(10,'(A)') '# x  c_true  c_model  c_analysis'
   do i = 1, nx
      write(10,'(4F12.6)') x(i), c_true(nt,i), c_model(nt,i), c_analysis(nt,i)
   end do
   close(10)
   write(*,*) 'Arquivo "oi_output.dat" gerado para plot externo.'
end program advec_diff_OI
```

```
import numpy as np
import matplotlib.pyplot as plt

# ----------------------------
# Parâmetros do modelo
# ----------------------------
nx = 50
nt = 50
dx = 1.0 / nx
dt = 0.001
u = 1.0      # velocidade de advecção
kappa = 0.01  # difusão

x = np.linspace(0, 1, nx)

# Condição inicial (pulso gaussiano)
c0 = np.exp(-200 * (x - 0.3)**2)

# Arrays de solução
c_true = np.zeros((nt, nx))
c_model = np.zeros((nt, nx))
c_analysis = np.zeros((nt, nx))

c_true[0] = c0.copy()
c_model[0] = c0.copy()

# ----------------------------
# Simulação da verdade
# ----------------------------
def step_advec_diff(c):
    """Passo explícito advecção-difusão 1D"""
    c_new = c.copy()
    for i in range(1, nx-1):
        adv = -u * (c[i] - c[i-1]) / dx
        diff = kappa * (c[i+1] - 2*c[i] + c[i-1]) / dx**2
        c_new[i] = c[i] + dt * (adv + diff)
    return c_new

for n in range(1, nt):
    c_true[n] = step_advec_diff(c_true[n-1])
```

```
# ----------------------------
# Modelo com erro inicial
# ----------------------------
c_model[0] = c0 * 0.8  # background piorado
for n in range(1, nt):
    c_model[n] = step_advec_diff(c_model[n-1])

# ----------------------------
# Observações (a cada 10 passos)
# ----------------------------
obs_idx = [10, 25, 40]  # pontos onde há observações na grade
obs_error = 0.05
y_obs = c_true[:, obs_idx] + np.random.normal(0, obs_error, (nt, len(obs_idx)))

# ----------------------------
# Interpolação Ótima
# ----------------------------
model_error = 0.1
gain = model_error / (model_error + obs_error)

c_analysis[:] = c_model[:]

for n in range(nt):
    for j, idx in enumerate(obs_idx):
        c_analysis[n, idx] = (1 - gain) * c_model[n, idx] + gain * y_obs[n, j]

# ----------------------------
# Plot comparativo
# ----------------------------
plt.figure(figsize=(10,6))
plt.plot(x, c_true[-1], 'k-', lw=2, label="Verdade (final)")
plt.plot(x, c_model[-1], 'r--', lw=2, label="Modelo (background)")
plt.plot(x, c_analysis[-1], 'b-', lw=2, label="Análise (OI)")
plt.scatter(x[obs_idx], y_obs[-1], c='green', s=60, marker='o', label="Observações")
plt.xlabel("x")
plt.ylabel("Concentração")
plt.title("Assimilação por Interpolação Ótima em Advecção-Difusão 1D")
plt.legend()
plt.grid()
plt.show()
```

Paulo Yoshio Kubota

Usa uma **matriz de covariância de fundo B não-diagonal** (com correlação espacial gaussiana) na Interpolação Ótima (OI).

O algoritmo usado:

•constrói $\mathrm{B}(nx, nx)$ com correlação gaussiana: $B_{i,j} = \sigma_b^2 e^{\left[-\frac{(x_i - x_j)^2}{2L^2}\right]}$;

•monta H implicitamente usando os índices de observação (ou seja, só pega linhas/colunas relevantes de $\mathrm{B}(nx, nx)$);

•calcula $S = HBH^T + R$ (matriz de inovação, nobs×nobs) e inverte S (Gauss-Jordan — suficiente para nobs pequenos/medianos);

•calcula o ganho $K = BH^T S^{-1}$(nx×nobs);

•atualiza a análise no tempo n : $x_a = x_b + K(y - Hx_b)$.

- geração de ruído gaussiano (Box–Muller).

Modelo Base Adveccao-difusao

```
!=====================================================================
! Programa : advec_diff_OI_B.f90
! Descrição: Adveccao-difusao 1D + Assimilação por OI com B nao-diagonal
!          B: covariancia gaussiana espacial
! Padrão   : Fortran 2023
! Autor    : convertido por GPT-5 (exemplo)
!=====================================================================

module advec_diff_mod
   use iso_fortran_env, only: real64
   implicit none
contains
   pure function step_advec_diff(c, nx, dx, dt, u, kappa) result(c_new)
      integer, intent(in) :: nx
      real(real64), intent(in) :: dx, dt, u, kappa
      real(real64), intent(in) :: c(nx)
      real(real64) :: c_new(nx)
      integer :: i
      real(real64) :: adv, diff

      c_new = c
      do concurrent (i = 2:nx-1)
         adv  = -u * (c(i) - c(i-1)) / dx
         diff =  kappa * (c(i+1) - 2.0_real64*c(i) + c(i-1)) / dx**2
         c_new(i) = c(i) + dt * (adv + diff)
      end do
   end function step_advec_diff
end module advec_diff_mod
```

```
module lin_alg_mod
   use iso_fortran_env, only: real64
   implicit none
contains
   function invert_matrix(A, n) result(Ainv)
      ! Gauss-Jordan inversion. Assumes A is non-singular.
      integer, intent(in) :: n
      real(real64), intent(in) :: A(n,n)
      real(real64) :: Ainv(n,n)
      real(real64), allocatable :: M(:,:)
      integer :: i, j, k
      real(real64) :: pivot, fac
      allocate(M(n,n))
      M = A
      Ainv = 0.0_real64
      do i = 1, n
         Ainv(i,i) = 1.0_real64
      end do
      do i = 1, n
         pivot = M(i,i)
         if (abs(pivot) < 1.0e-14_real64) then
            ! try to swap with a lower row
            do k = i+1, n
               if (abs(M(k,i)) > abs(pivot)) then
                  M([i,k],:) = M([k,i],:)
                  Ainv([i,k],:) = Ainv([k,i],:)
                  pivot = M(i,i)
                  exit
               end if
            end do
         end if
         if (abs(pivot) < 1.0e-14_real64) then
            stop "invert_matrix: singular matrix or pivot ~ 0"
         end if
```

```
         ! normalize row i
         M(i,:) = M(i,:) / pivot
         Ainv(i,:) = Ainv(i,:) / pivot
         ! eliminate other rows
         do j = 1, n
            if (j /= i) then
               fac = M(j,i)
               if (abs(fac) > 0.0_real64) then
                  M(j,:) = M(j,:) - fac * M(i,:)
                  Ainv(j,:) = Ainv(j,:) - fac * Ainv(i,:)
               end if
            end if
         end do
      end do

      deallocate(M)
   end function invert_matrix


   pure function matvec_mul(A, x, n, m) result(y)
      ! y = A * x where A(n,m), x(m)
      integer, intent(in) :: n, m
      real(real64), intent(in) :: A(n,m), x(m)
      real(real64) :: y(n)
      integer :: i, j
      y = 0.0_real64
      do i = 1, n
         do j = 1, m
            y(i) = y(i) + A(i,j) * x(j)
         end do
      end do
   end function matvec_mul

end module lin_alg_mod
```

Paulo Yoshio Kubota

```
program advec_diff_OI_B
    use iso_fortran_env, only: real64
    use advec_diff_mod
    use lin_alg_mod
    implicit none

    ! Parameters
    integer, parameter      :: nx = 50, nt = 50
    real(real64), parameter :: dx = 1.0_real64 / nx, dt = 0.001_real64
    real(real64), parameter :: u = 1.0_real64, kappa = 0.01_real64

    ! State arrays
    real(real64), allocatable :: x(:)
    real(real64), allocatable :: c_true(:,:), c_model(:,:), c_analysis(:,:)
    real(real64), allocatable :: c0(:)

    ! Covariance / OI
    real(real64), allocatable :: B(:,:), B_sub(:,:), K(:,:), S(:,:), Sinv(:,:)
    real(real64), allocatable :: innov(:)
    real(real64), allocatable :: increment(:)

    ! Observations
    integer, PARAMETER        :: nobs=3 !observation number point
    integer, allocatable      :: obs_idx(:)
    real(real64), allocatable :: y_obs(:,:)
    real(real64) :: obs_error = 0.05_real64
    real(real64) :: model_error = 0.1_real64   ! sigma_b
    real(real64) :: Lcorr = 0.1_real64        ! comprimento de correlação
    real(real64) :: gain_diag
    ! temp & loop
    integer :: n, i, j
    real(real64) :: r1, r2
```

```
! --------------------------
   ! Allocate
   ! --------------------------
   allocate(x(nx), c_true(nt, nx), c_model(nt, nx), c_analysis(nt, nx))
   allocate(c0(nx))
   ! definir observacoes (mesmos índices do exemplo)
   allocate(obs_idx(nobs))
   allocate(y_obs(nt, nobs))
   ! B e matrizes auxiliares
   allocate(B(nx, nx))
   allocate(B_sub(nx, nobs))
   allocate(K(nx, nobs))
   allocate(S(nobs, nobs))
   allocate(Sinv(nobs, nobs))
   allocate(innov(nobs))
   allocate(increment(nx))


   ! --------------------------
   ! Grade e condicao inicial
   ! --------------------------
   do i = 1, nx
       x(i) = real(i-1, real64) / real(nx-1, real64)
       c0(i) = exp(-200.0_real64 * (x(i) - 0.3_real64)**2)
   end do
   !
   ! value of the observation
   !
   obs_idx = [11, 26, 41]   ! Fortran 1-based

   c_true(1,:) = c0
   c_model(1,:) = 0.8_real64 * c0
```

```
! ---------------------------
! Simulacao (verdade e modelo)
! ---------------------------
do n = 2, nt
   c_true (n,:) = step_advec_diff(c_true(n-1,:), nx, dx, dt, u, kappa)
   c_model(n,:) = step_advec_diff(c_model(n-1,:), nx, dx, dt, u, kappa)
end do

! ---------------------------
! Construir B (covariancia gaussiana)
! B_ij = sigma_b^2 * exp( - (x_i - x_j)^2 / (2 L^2) )
! ---------------------------
do i = 1, nx
   do j = 1, nx
      B(i,j) = model_error**2 * exp( - (x(i)-x(j))**2 / (2.0_real64 * Lcorr**2) )
   end do
end do

! ---------------------------
! Gerar observacoes (ruido Gaussiano) usando Box-Muller
! ---------------------------
call random_seed()
do n = 1, nt
   do i = 1, nobs
      call random_number(r1)
      call random_number(r2)
      if (r1 <= 0.0_real64) r1 = 1.0e-12_real64
      ! Box-Muller ? z ~ N(0,1)
      y_obs(n,i) = c_true(n, obs_idx(i)) + obs_error * sqrt(-2.0_real64*log(r1)) * cos(2.0_real64*acos(-1.0_real64)*r2)
   end do
end do
```

- $B_{i,j} = \sigma_b^2 e^{\left[-\frac{(x_i - x_j)^2}{2L^2}\right]};$

- geração de ruído gaussiano (Box–Muller).

```
! ----------------------------
! Preencher B_sub = B(:, obs_idx) (nx x nobs)
! ----------------------------
do i = 1, nx
   do j = 1, nobs
      B_sub(i,j) = B(i, obs_idx(j))
   end do
end do
! ----------------------------
! Calcular S = H B H^T + R  (nobs x nobs)
! Como H seleciona indices, HBHT = B(obs_idx, obs_idx)
! R = obs_error^2 * I
! ----------------------------
do i = 1, nobs
   do j = 1, nobs
      S(i,j) = B(obs_idx(i), obs_idx(j))
   end do
end do
do i = 1, nobs
   S(i,i) = S(i,i) + obs_error**2
end do
! Inverter S
Sinv = invert_matrix(S, nobs)
! ----------------------------
! Calcular ganho K = B_sub * Sinv   (nx x nobs) * (nobs x nobs) -> (nx x nobs)
! ----------------------------
do i = 1, nx
   do j = 1, nobs
      K(i,j) = 0.0_real64
      do n = 1, nobs
         K(i,j) = K(i,j) + B_sub(i,n) * Sinv(n,j)
      end do
   end do
end do
```

•monta H implicitamente usando os índices de observação (ou seja, só pega linhas/colunas relevantes de $\mathrm{B}(nx, nx)$);

•calcula $S = HBH^T + R$ (matriz de inovação, nobs×nobs)

inverte S (Gauss-Jordan — suficiente para nobs pequenos/medianos);

•calcula o ganho $K = BH^T S^{-1}$(nx×nobs);

```
! ---------------------------
! Aplicar OI em cada passo temporal:
! xa = xb + K (y - H xb)
! H xb e apenas xb nos indices observados
! ---------------------------
c_analysis = c_model
do n = 1, nt
   do i = 1, nobs
      innov(i) = y_obs(n,i) - c_model(n, obs_idx(i))
   end do

   ! increment = K * innov -> vetor nx

   increment = matvec_mul(K, innov, nx, nobs)

   ! atualizar análise
   do i = 1, nx
      c_analysis(n,i) = c_model(n,i) + increment(i)
   end do
end do
! ---------------------------
! Saida: imprimir alguns valores e salvar arquivo para plot
! ---------------------------
write(*,'(A)') '============================================================'
write(*,'(A)') '  OI com B nao-diagonal (covariancia gaussiana espacial)'
write(*,'(A)') '============================================================'
write(*,'(A,F8.4)') 'sigma_b (model_error) = ', model_error
write(*,'(A,F8.4)') 'Lcorr = ', Lcorr
write(*,'(A,F8.4)') 'obs_error = ', obs_error
write(*,'(A)') 'Inovacao e analise final nos pontos observados:'
do i = 1, nobs
   write(*,'(A,I3, A, F8.4, A, F8.4)') ' idx=', obs_idx(i), &
      '   obs=', y_obs(nt,i), '   analise=', c_analysis(nt, obs_idx(i))
end do
```

- atualiza a análise no tempo n : $x_a = x_b + K(y - Hx_b)$.

```
   write(*,'(A)') 'Arquivo "oi_B_output.dat" gerado para plot externo.'
   write(*,'(A)') '============================================================'

   open(unit=11, file='oi_B_output.dat', status='replace')
   write(11,'(A)') '# x  c_true  c_model  c_analysis'
   do i = 1, nx
      write(11,'(4F12.6)') x(i), c_true(nt,i), c_model(nt,i), c_analysis(nt,i)
   end do
   close(11)

   ! liberar
   deallocate(B, B_sub, K, S, Sinv)
   deallocate(x, c_true, c_model, c_analysis, c0, y_obs, obs_idx)
   deallocate(innov)
   deallocate(increment)

end program advec_diff_OI_B
```

Conclusão do Módulo 1

A assimilação de dados tem como **objetivo melhorar a previsão** e a compreensão do estado da atmosfera.

O uso de técnicas como interpolação ótima e filtragem de Kalman ajuda a ajustar o modelo com base em observações.